



Ahmed Irshad Hussain

sardarahmed.irshad2@gmail.com

© COPYRIGHT ALL RIGHTS RESERVED



CLOUD DATA ENGINEERING:

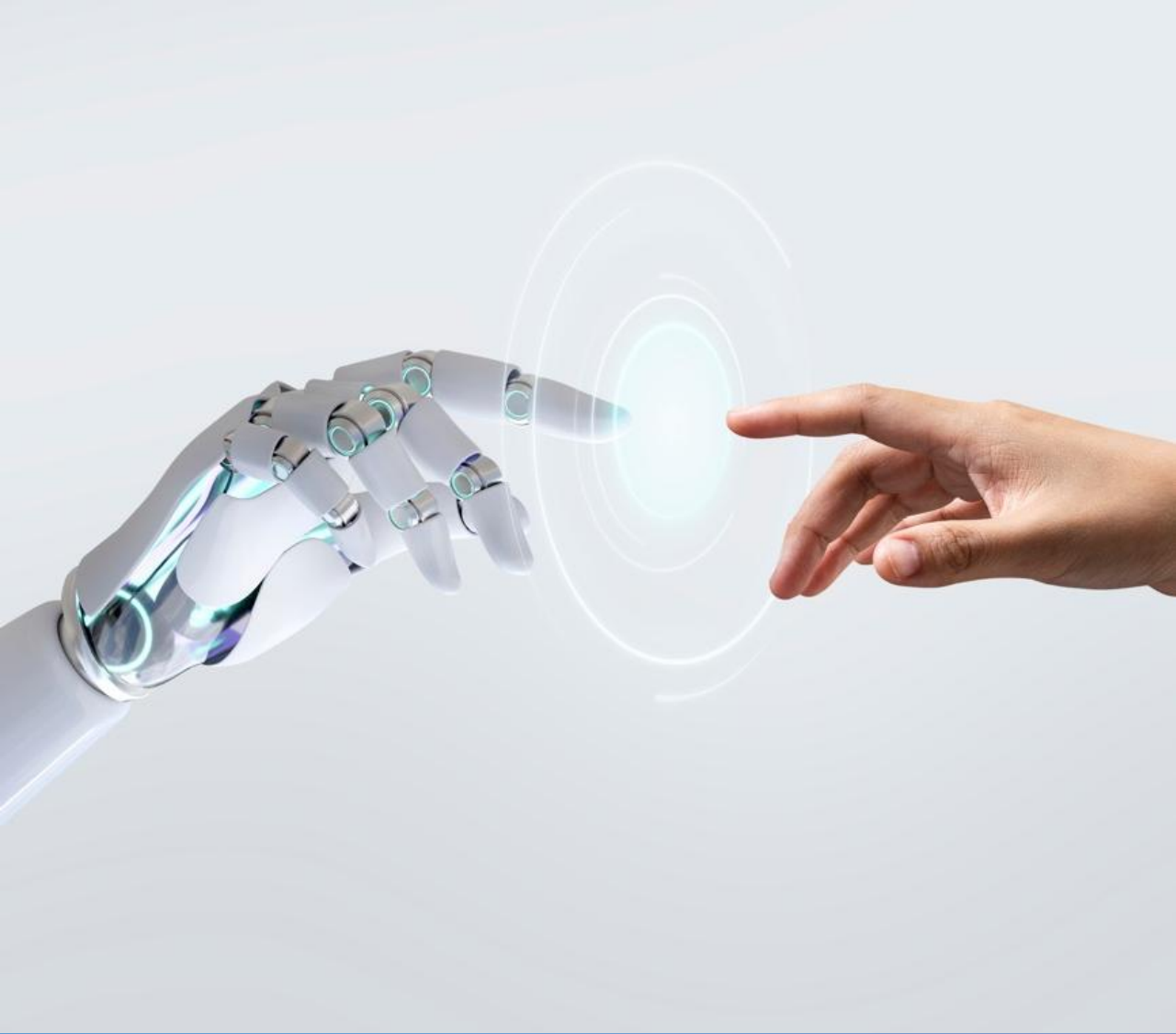
MICROSOFT FABRIC

Data Modeling, Power BI Relationships

DAX & Analytics, Enterprise BI

Reporting

By: Ahmed Irshad Hussain



Data Engineering: Microsoft Fabric Star Schema

First Understand the BIG Picture

Right now, my Gold layer probably looks like:

category	total_revenue
Electronics	500000
Clothing	90000

This is simple aggregation.

Good for learning.

But not enterprise-grade modeling.

Problem with Current Gold Layer

My current Gold table:

- mixes dimensions and metrics
- not scalable
- limited analytics
- weak for large BI systems
- difficult for advanced slicing

Enterprise Solution = STAR SCHEMA

Instead of ONE big table , I'll split data into:
Fact Tables + Dimension Tables

Core Idea

Fact Table,
Stores:

- measurable business events
- numbers
- metrics

Example

- revenue
- quantity
- sales

Dimension Tables,
Store:

- descriptive information

Example:

- customer
- product
- category
- Date



My Project Transformation

Currently, I have “silver_sales”
contains:

- transaction_id
- category
- revenue
- quantity
- Date

After Star Schema

Fact table = fact_sales

Contains,
sale_id, product_id, date_id, revenue, quantity

Notice,

- Mostly numeric
- Foreign keys
- Business measurement

Dimension Tables

dim_product:

product_id & category

dim_date:

dim_id, month, quarter, year

Microsoft Fabric

STEP 1: Creating Fact Table

- First load silver table
- Then I'll use row_number() with window function
- It will generate sequential id

```

1  from pyspark.sql.window import Window
2  from pyspark.sql.functions import row_number
3
4  # Read Silver Layer
5  df = spark.table("silver_sales")
6
7  # Window specification
8  window_spec = Window.orderBy("date")
9
10 # Create Fact Table with Sequential IDs
11 fact_sales = df.withColumn(
12     "sale_id",
13     row_number().over(window_spec)
14 ).select(
15     "sale_id",
16     "date",
17     "category",
18     "revenue",
19     "quantity"
20 )
21
22 # Save Fact Table
23 fact_sales.write.format("delta") \
24     .mode("overwrite") \
25     .option("overwriteSchema", "true") \
26     .saveAsTable("fact_sales")

```

✓ - Command executed in 7 sec 341 ms by Ahmed Fabric on 5/8/2026, 4:55:05 PM



Verifying!
As you see, there are unique
sale_id generated for every
sale

123 sale_id	📅 date	ABC category	123 revenue
350	2022-12-17	ELECTRONICS	1610
351	2022-12-18	BAGS	760
352	2022-12-19	CLOTHING	410
353	2022-12-20	ELECTRONICS	2410
354	2022-12-21	ELECTRONICS	4210
355	2022-12-22	SHOES	970
356	2022-12-23	CLOTHING	510

TIP

Don't use monotonically, as it generates IDs non-sequentially

```
fact_sales = df.select(  
    monotonically_increasing_id().alias("sale_id"),  
    "date",  
    "category",  
    "revenue",  
    "quantity"
```

	12L sale_id	...	date	ABC category
127	126		2022-06-02	ELECTRONICS
128	127		2022-05-07	ELECTRONICS
129	128		2022-06-03	ELECTRONICS
130	8589934592		2022-02-18	CLOTHING
131	8589934593		2022-01-15	CLOTHING
132	8589934594		2022-02-24	CLOTHING

Microsoft Fabric

STEP 2: Creating dim_product

- Right now, my fact table contains:
category
- But in Star Schema:
Descriptions belong in Dimensions.
- So, I'll separate product/category info into:
dim_product

```
1 from pyspark.sql.functions import dense_rank
2 from pyspark.sql.window import Window
3
4 # Read Silver Layer
5 df = spark.table("silver_sales")
6
7 # Window for IDs
8 window_spec = Window.orderBy("category")
9
10 # Create Product Dimension
11 dim_product = df.select("category").distinct() \
12     .withColumn(
13         "product_id",
14         dense_rank().over(window_spec)
15     ) \
16     .select(
17         "product_id",
18         "category"
19     )
20
21 # Save Dimension Table
22 dim_product.write.format("delta") \
23     .mode("overwrite") \
24     .option("overwriteSchema", "true") \
25     .saveAsTable("dim_product")
```

✓ 10 sec - Command executed in 9 sec 836 ms by Ahmed Fabric on 5/8/2026, 10:16:04 PM

> Spark jobs (9 of 9 succeeded) Resources

Important Note!

Why I used dense rank instead of row number like my previous cell?
Because, dimension tables represent grouped business entities rather than individual transaction rows.

For example:

Category	Sub-category
Clothing	Pants
Clothing	Shirts
Shoes	Boots

If row_number() were used directly on transactional rows, each row could receive a different identifier even though they belong to the same business category. Using dense_rank() ensures that all records belonging to the same category receive the same grouped identifier structure after applying distinct business values.

So, if I'll use row number, it will give different id to clothing pants and shirts but technically these both are clothing so id should be same because of the category as it will help in producing accurate business analytics.

Microsoft Fabric

STEP 3: Creating dim_date

We'll extract:

- Day
- Month
- Year
- Quarter

From sales dates.

Why this is powerful

- Now Power BI can:
- filter by quarter
- trend by month
- compare yearly revenue
- drill down over time

VERY efficiently.

```
1 from pyspark.sql.functions import \
2     year, month, quarter, dayofmonth
3 from pyspark.sql.window import Window
4 from pyspark.sql.functions import dense_rank
5
6 # Read Silver Layer
7 df = spark.table("silver_sales")
8
9 # Create Date Dimension
10 dim_date = df.select("date").distinct() \
11     .withColumn("day", dayofmonth("date")) \
12     .withColumn("month", month("date")) \
13     .withColumn("quarter", quarter("date")) \
14     .withColumn("year", year("date"))
15
16 # Generate Date IDs
17 window_spec = Window.orderBy("date")
18
19 dim_date = dim_date.withColumn(
20     "date_id",
21     dense_rank().over(window_spec)
22 ).select(
23     "date_id",
24     "date",
25     "day",
26     "month",
27     "quarter",
28     "year"
29 )
30
31 # Save Date Dimension
32 dim_date.write.format("delta") \
33     .mode("overwrite") \
34     .option("overwriteSchema", "true") \
35     .saveAsTable("dim_date")
```

[1] ✓ 38 sec - Session ready in 11 sec 974 ms. Command executed in 26 sec 539 ms by Ahmed Fabric on 5/8/2026, 10:40:04 PM

> 📋 Spark jobs (15 of 15 succeeded) 📖 Resources

This will create,
date_id, date, day, month, quarter, year

Verifying

dim_date

Showing 361 rows

Table view 

	123 date_id	 date	123 day	123 month	123 quarter	123 year
1	1	2022-01-01	1	1	1	2022
2	2	2022-01-02	2	1	1	2022
3	3	2022-01-03	3	1	1	2022
4	4	2022-01-04	4	1	1	2022
5	5	2022-01-05	5	1	1	2022
6	6	2022-01-06	6	1	1	2022
7	7	2022-01-07	7	1	1	2022
8	8	2022-01-08	8	1	1	2022
9	9	2022-01-09	9	1	1	2022
10	10	2022-01-10	10	1	1	2022
11	11	2022-01-11	11	1	1	2022
12	12	2022-01-12	12	1	1	2022
13	13	2022-01-13	13	1	1	2022
14	14	2022-01-14	14	1	1	2022
15	15	2022-01-15	15	1	1	2022

Microsoft Fabric

STEP 4:

Right now, my fact table still contains:

- Date
- Category

Instead of:

- date_id
- product_id

That means:

- relationships are conceptual right now
- not fully dimensional yet

```
1 # Read tables
2 silver_df = spark.table("silver_sales")
3 dim_product = spark.table("dim_product")
4 dim_date = spark.table("dim_date")
5
6 # Join with Product Dimension
7 fact_sales = silver_df.join(
8     dim_product,
9     on="category",
10    how="left"
11 )
12
13 # Join with Date Dimension
14 fact_sales = fact_sales.join(
15     dim_date,
16     on="date",
17     how="left"
18 )
19
20 # Select final fact table columns
21 fact_sales = fact_sales.select(
22     "product_id",
23     "date_id",
24     "revenue",
25     "quantity"
26 )
27
28 # Save final fact table
29 fact_sales.write.format("delta") \
30     .mode("overwrite") \
31     .option("overwriteSchema", "true") \
32     .saveAsTable("fact_sales")
```

[1] ✓ 55 sec - Command executed in 38 sec 979 ms by Ahmed Fabric on 5/8/2026, 11:35:14 PM

> ⚙️ Spark jobs (20 of 20 succeeded) 📄 Resources 📖 Log

Verifying

	123 product_id	123 date_id	123 revenue	123 quantity
1	5	26	970	12
2	5	13	810	10
3	5	44	650	8
4	5	82	970	12
5	5	52	410	5
6	5	72	1210	15
7	5	352	970	12
8	5	313	650	8
9	5	342	1210	15
10	5	293	970	12
11	5	321	970	12
12	5	283	1210	15
13	5	232	970	12
14	5	192	650	8
15	5	201	970	12
16	5	262	970	12
17	5	253	650	8
18	5	222	1210	15
19	5	102	650	8
20	5	112	970	12
21	5	172	970	12

Next Steps

Power BI Relationship Modeling & Star Schema

Relationship modeling connects your data tables so they interact correctly inside dashboards. By upgrading from a single, bulky "flat table" to a Star Schema, your data becomes scalable, maintainable, and analytics-ready for enterprise reporting.

Fact vs. Dimension Tables

Fact tables (fact_sales) store massive, continuously growing transactional numbers like revenue and quantity.

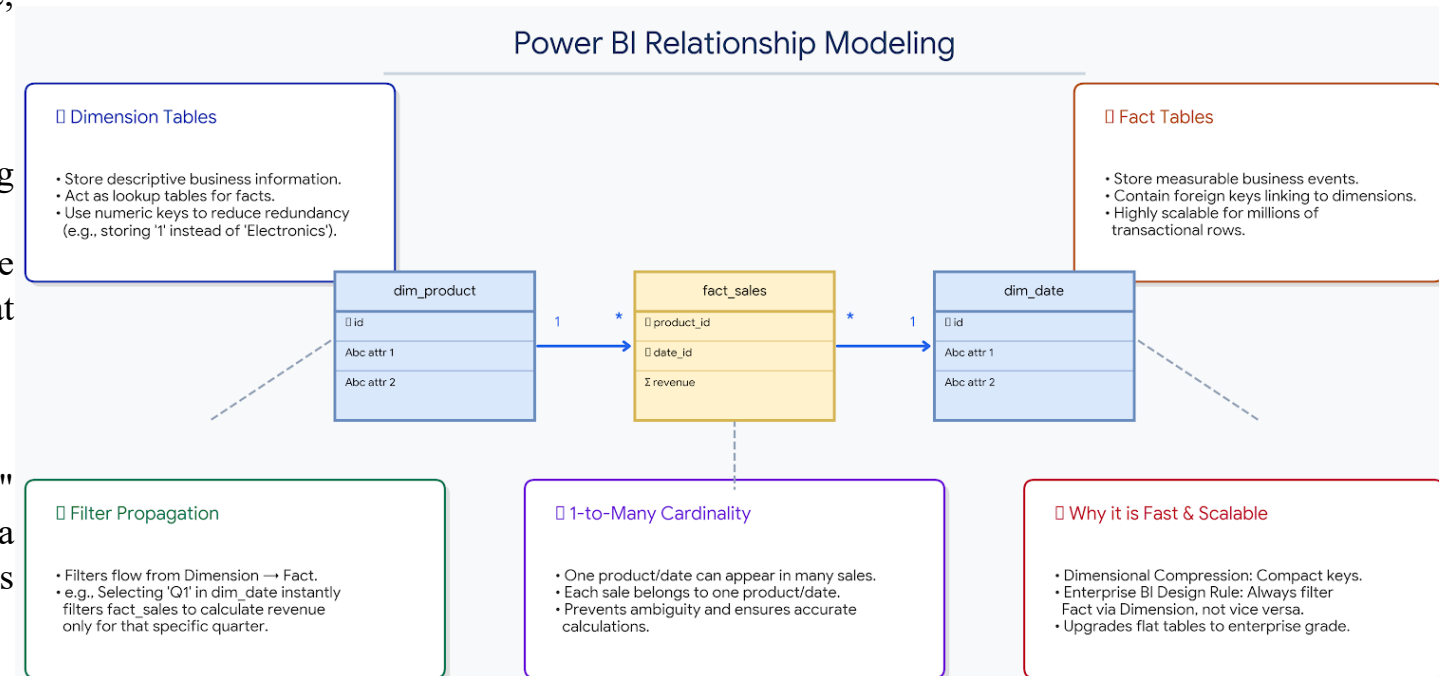
Dimension tables (dim_product, dim_date) act as small, descriptive lookup dictionaries (storing product names or specific quarters) that the fact table references using compact ID keys.

One-to-Many Cardinality

Tables are linked using matching ID keys to create a "One-to-Many" relationship. This means a single unique product or specific date in a dimension table can connect to thousands of individual sales transactions within the massive fact table.

Filter Propagation & Performance

In an enterprise design, filters always flow in one direction: from Dimension to Fact. By filtering a small dimension table first, Power BI instantly pushes that filter down to the fact table, dramatically reducing query complexity and speeding up your dashboard.



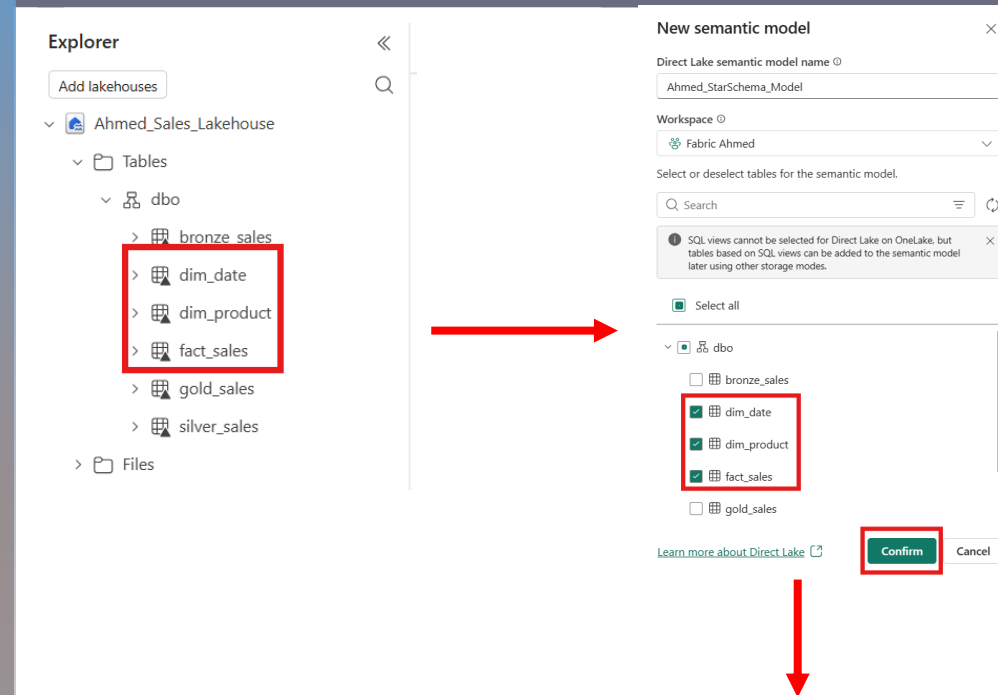
Microsoft Fabric

Power BI Relationship Modeling

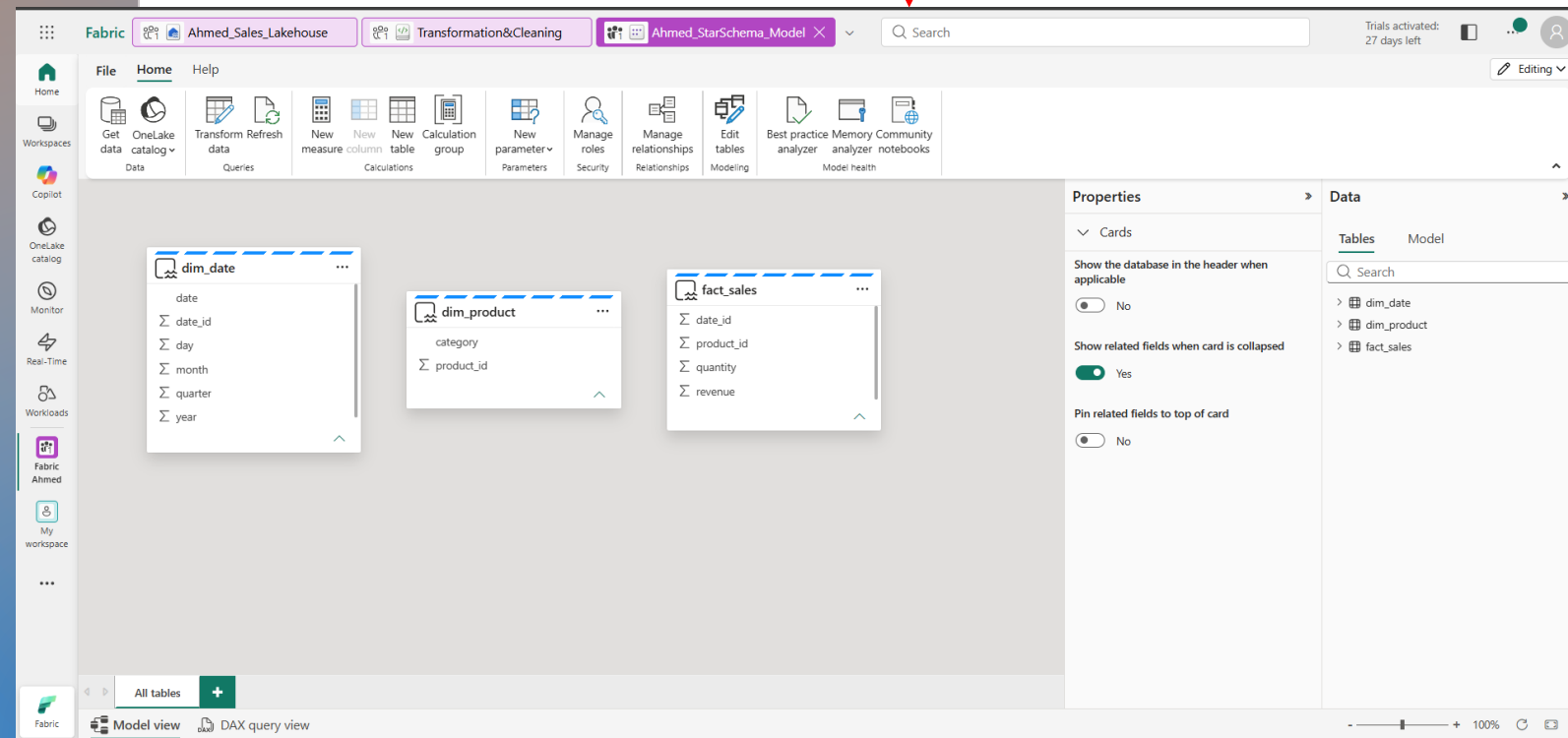
STEP 5: Create New Semantic Model

My old semantic model was built on gold layer; I need to build my new semantic model on,

- Fact tables
- Dim product
- Dim date



1. Go back to lakehouse
2. Create new semantic model
3. Inside, lakehouse: select (fact_tables, dim_product, dim_date)
4. Click Create



Microsoft Fabric

Power BI Relationship Modeling

STEP 6: Create Relationships

RELATIONSHIP 1:

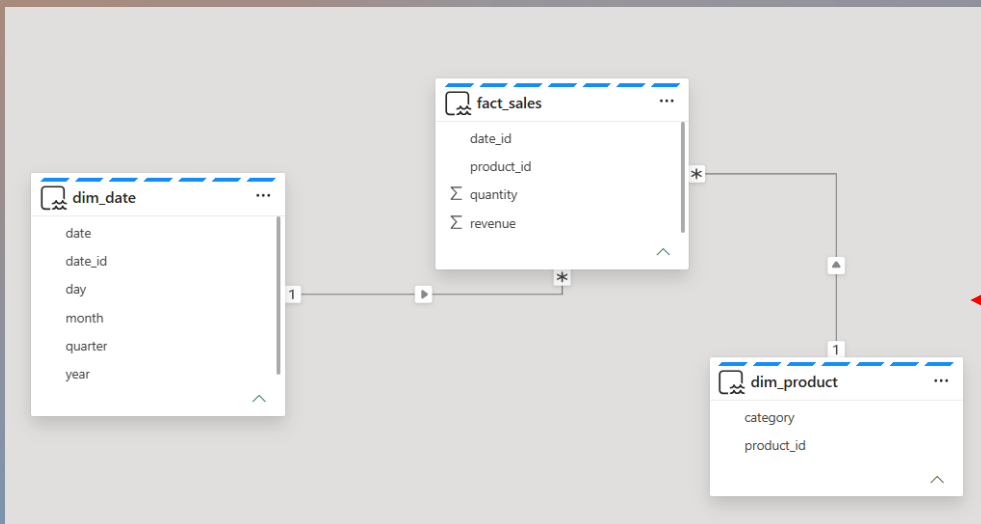
dim_product.product_id → fact_sales.product_id

Settings: One-to-Many, Single direction

RELATIONSHIP 2:

dim_date.date_id → fact_sales.date_id

Settings: One-to-Many, Single direction



←

New relationship

×

Select tables and columns that are related.

From table

dim_product

category product_id

A preview of this table isn't available

To table

fact_sales

date_id product_id quantity revenue

A preview of this table isn't available

Cardinality

One to many (1:*)

Cross-filter direction

Single

☒ Make this relationship active

☐ Apply security filter in both directions

☐ Assume referential integrity [Learn more](#)

Direct Lake tables don't show data previews or validate relationships cardinality and cross-filter direction. Cardinality is determined by table row count and single cross-filter direction is always populated but these properties may need to be changed manually to reflect the relationship correctly.

Save Cancel

Do this same for date

From table = dim_date

To table = fact_sales

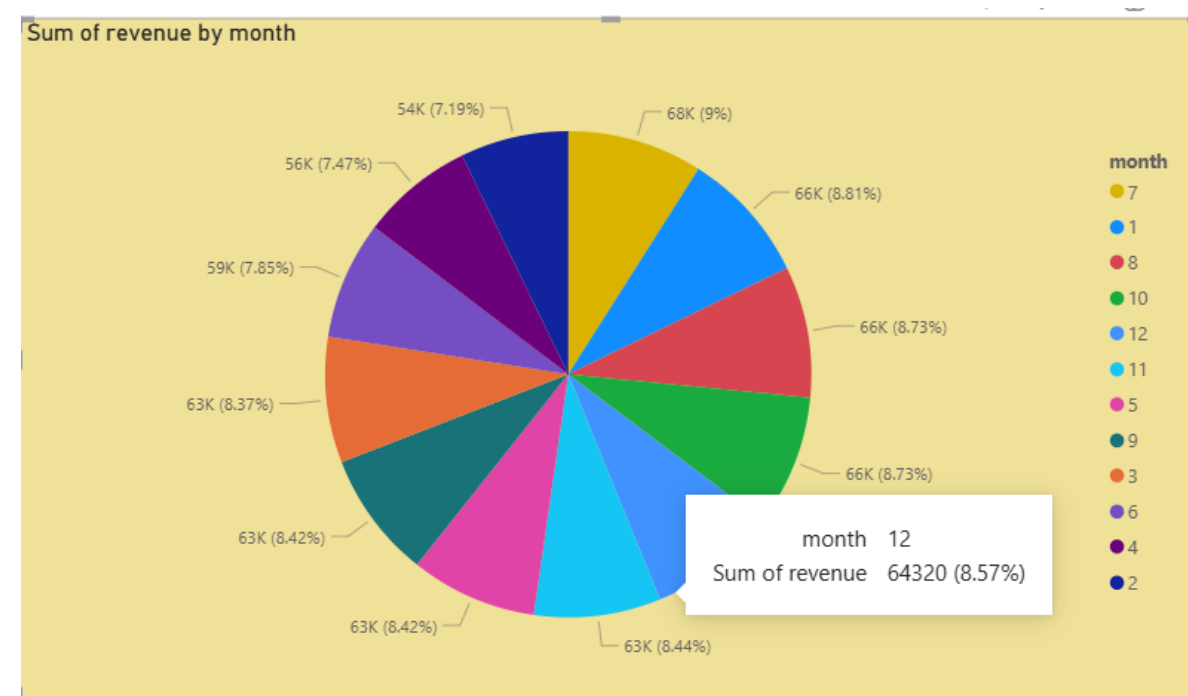
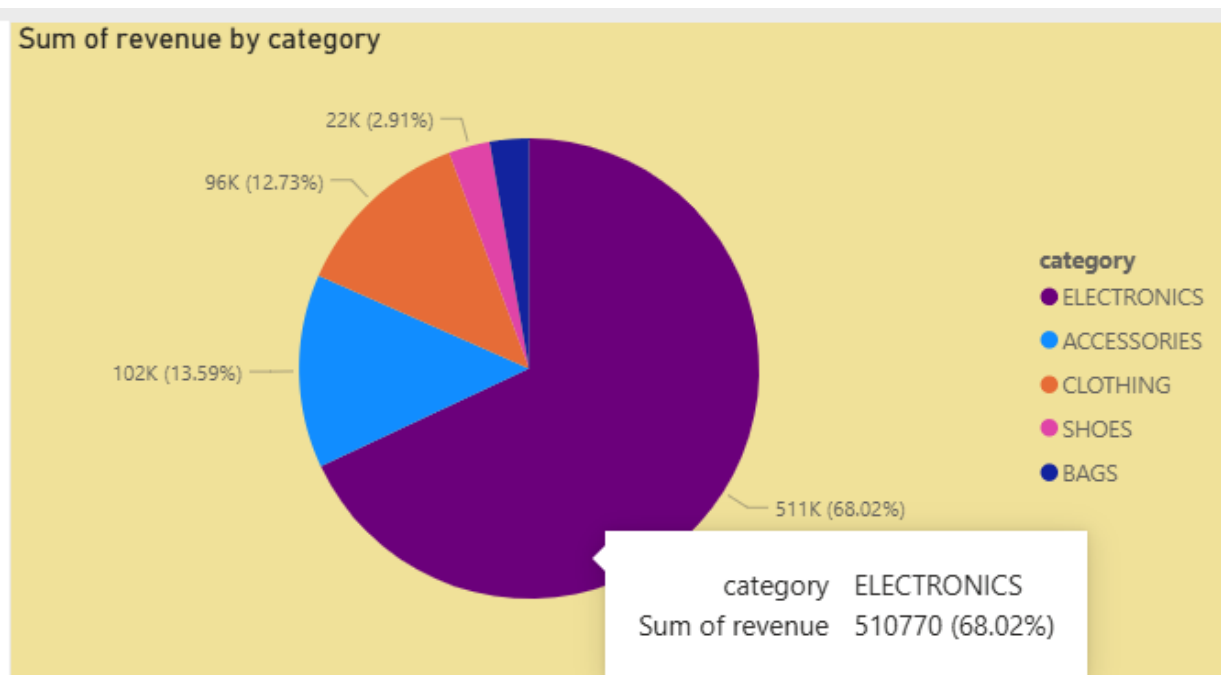
Cardinality = one to many

Cross direction = single

Manage relationships

+ New relationship				Edit	Delete	Filter
☐ From: table (column) ↑	Relationship	To: table (column)	Status			
☐ fact_sales (date_id)	* ← 1	dim_date (date_id)	Active	...		
☐ fact_sales (product_id)	* ← 1	dim_product (product_id)	Active	...		

Unlike flat tables:
 NOW dimensions control filtering.
 This is:
 true dimensional analytics which is how enterprise BI systems are designed



BI Engineering

- Star Schema
- Fact Tables
- Dimension Tables
- Relationships
- One-to-Many Cardinality
- Single Direction Filtering
- Semantic Modeling

Reporting Layer

- Dynamic filtering
- Relationship-based visuals
- Enterprise analytical structure

SNOWFLAKE

CONCEPT ONLY



FIRST UNDERSTAND THE DIFFERENCE

Star Schema

Dimensions are flat, denormalized & simple.

Example:

Product_id	category
1	Electronics
2	Clothing

Very fast for BI.

Snowflake Schema

Dimensions become normalized & split into sub-dimensions

Example:

Dim_product:

Product_id	Category_id
------------	-------------

Dim_category:

Category_id	Category_name
-------------	---------------

Now dimensions themselves are connected.

Looks like snowflake branches

WHY SNOWFLAKE EXISTS

Because in enterprise systems, Dimensions can become HUGE.

Example:

- products
- geography
- customers
- suppliers

Instead of repeating:

- Electronics
- Electronics
- Electronics

inside dimension,

we normalize them.

STAR VS. SNOWFLAKE:

Feature	Star Schema	Snowflake Schema
Structure	Denormalized	Normalized
Performance	Faster	Slightly slower
Complexity	Simple	More complex
Storage	More redundancy	Less redundancy
BI Usage	Very common	Enterprise-scale systems

DAX – Data Analysis Expressions

What is DAX?

DAX (Data Analysis Expressions) is a formula language used in:

- Power BI
- SSAS Tabular
- Excel Power Pivot

It is used to:

- Create measures
- Create calculated columns
- Create calculated tables
- Perform advanced analytics

DAX is a formula language used to calculate values over data models in Power BI.

POWER BI



DAX

DAX – Data Analysis Expressions

Why DAX is Powerful?

With DAX you can calculate:

- Total Sales
- Running Total
- Year-to-Date (YTD)
- Month-over-Month Growth
- Ranking
- Top N
- Profit Margin
- Dynamic filtering
- Time intelligence

That's enterprise BI.

POWER BI



DAX

DAX – Data Analysis Expressions

Core DAX Concepts!

1. Measure vs Calculated Column:

Measure	Calculated Column
Calculated at Runtime	Calculated at Data Load
Depends on Filter Context	Row by row
Lightweight	Increases model size

2. Row Context vs Filter Context:

Row Context: Exists in calculated columns.

NewAmount= Sales_Practice [Amount] * 1.1

Works row by row.

Filter Context: Created By Visuals, Slicers, Filters, RLS & Calculate Function.

If user selects Lahore, Measure automatically calculates only Lahore sales. This is Filter Context.

3. CALCULATE():

Most important function in DAX.

It modifies filter context.

Example:

Lahore Sales =

CALCULATE(

SUM(Sales_Practice[Amount]),

Sales_Practice[City] = "Lahore"

)

This forces filter to Lahore.

Microsoft Fabric

DAX

STEP 7: Create New Measures:

Create 3 measures,

- Total Revenue
- Total Quantity
- Average Revenue

WHY MEASURES ARE IMPORTANT?

Measures are:

- NOT physically stored
- calculated dynamically
- optimized in memory engine

This makes BI:

interactive and scalable

The screenshot illustrates the process of creating new measures in Microsoft Fabric. It shows the 'Home' tab with the 'New measure' button highlighted. A red arrow labeled '1' points to the 'New measure' button. Another red arrow labeled '2' points to the 'New measure' option in the context menu of the 'fact_sales' table. A third red arrow labeled '3' points to the 'fact_sales' table in the 'Data' pane, where the measures 'Average Revenue', 'Total Quantity', and 'Total Revenue' are listed. The 'Total Revenue' measure is highlighted with a red box. The 'Queries' pane shows the DAX formula for 'Total Revenue': `1 Total Revenue = SUM(fact_sales[revenue])`.

1. Clicked New Measure.
2. Right clicked on fact_sales and then select new measure.
3. Wrote query.
4. Repeated 3 times as I made 3 measures

Fabric



Ahmed_StarSchema_Dashboard



Ahmed_Sales_Lakehouse



Ahmed_StarSchema_Model



Search

File

View

Reading view

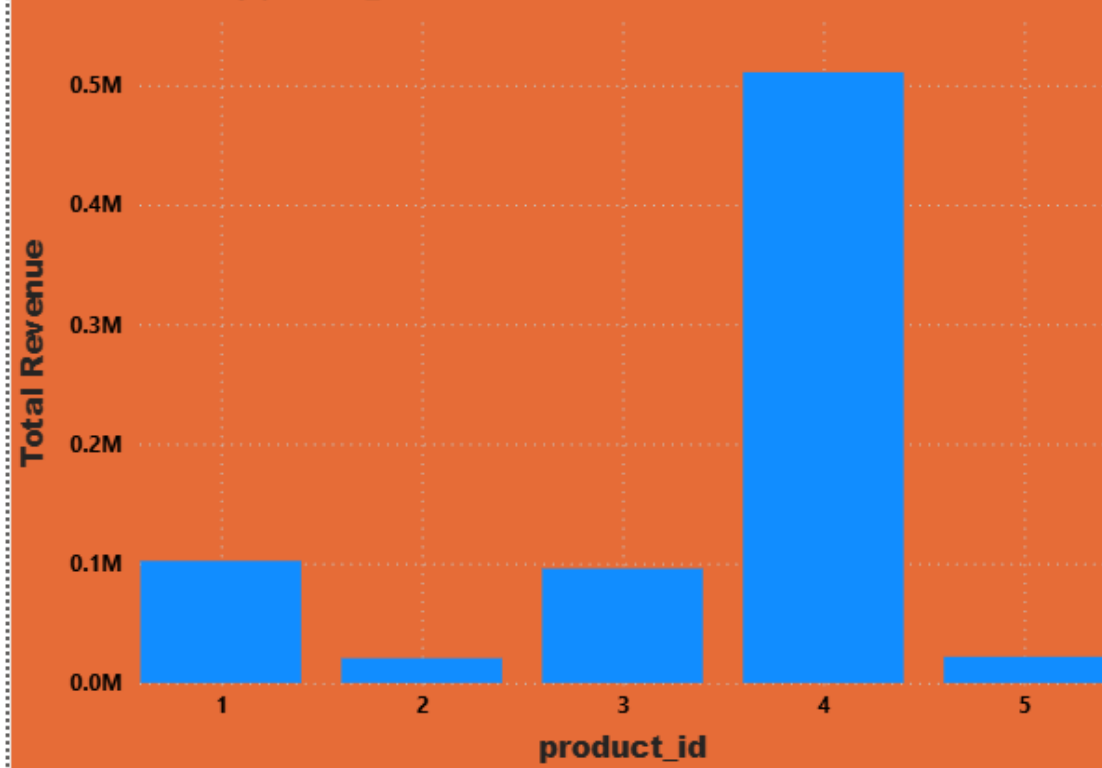
Mobile layout

Open semantic model

Copilot



Total Revenue by product_id



5K
Total Quantity

751K
Total Revenue

Microsoft Fabric

Time Intelligence in DAX

This is where dashboards become:

- executive-level
- business-oriented
- trend-aware

Because businesses ALWAYS ask:

- revenue growth
- monthly comparison
- YTD revenue
- quarterly trend
- previous month analysis

STEP 8: Create More Measure:

- Create YTD Measure.
- Create Growth % Measure.
- Create Previous Sales Measure

```

1 YTD Revenue =
2 TOTALYTD(
3     [Total Revenue],
4     dim_date[date]
5 )

```

WHAT THIS DOES

Suppose:

- Jan revenue = 100
- Feb revenue = 150
- Mar revenue = 200

Then:

Jan = 100

Feb = 250

March = 450

```

1 Previous Month Revenue =
2 CALCULATE(
3     [Total Revenue],
4     PREVIOUSMONTH(dim_date[date])
5 )

```

If current month = March

This measure calculates:

February revenue

VERY useful for comparisons.

```

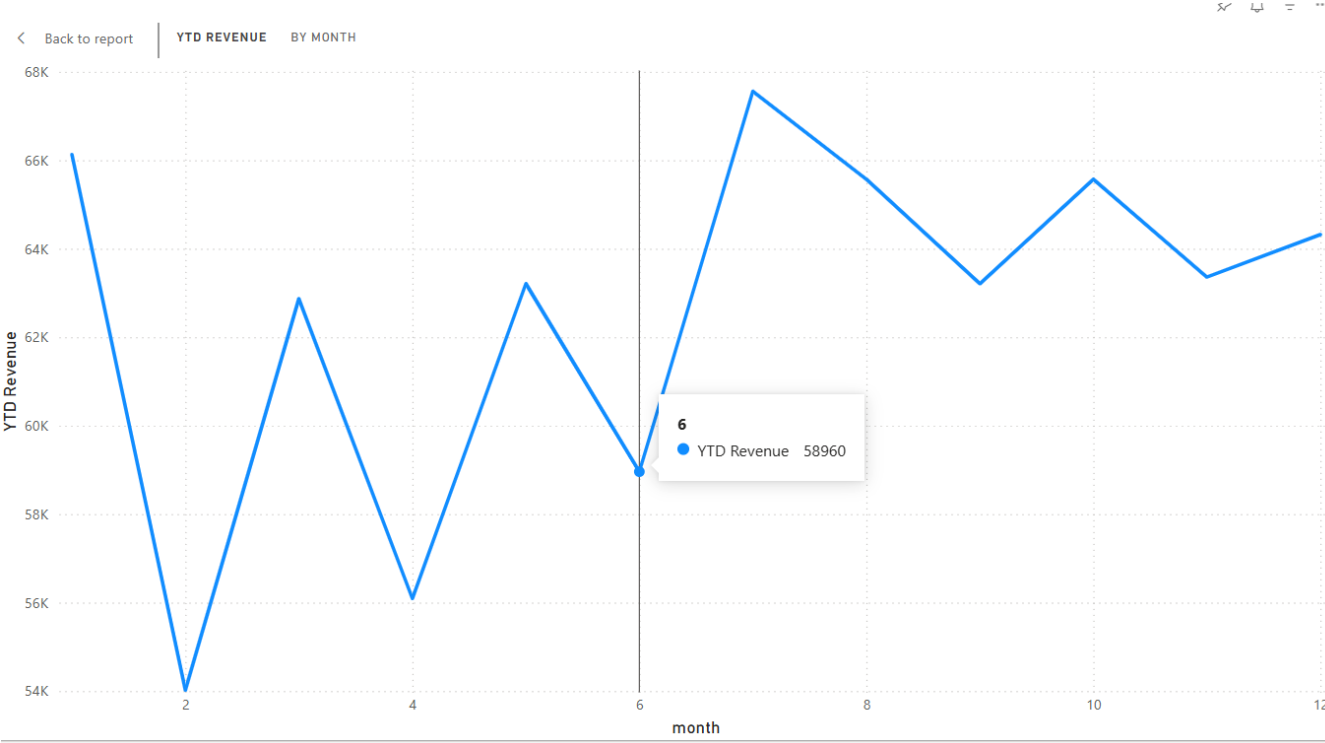
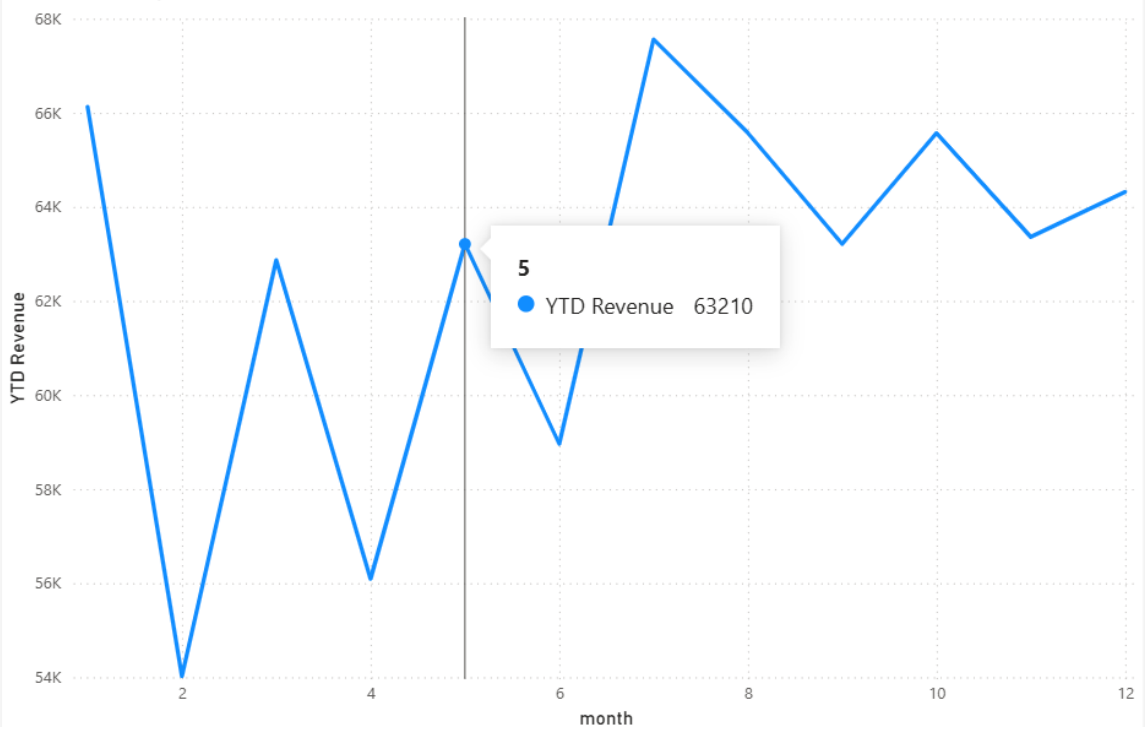
1 Revenue Growth % =
2 DIVIDE(
3     [Total Revenue] - [Previous Month Revenue],
4     [Previous Month Revenue]
5 )

```

month-over-month growth percentage



YTD Revenue by month



FLOWCHART

